

Experiment 9: A Search Algorithm*

Aim

To implement the A* Search algorithm in Python.

Objective

- To find the shortest path between two nodes.
- To use heuristic values for efficient searching.

Algorithm

1. Initialize open and closed lists.
2. Add the start node to the open list.
3. Select the node with minimum $f(n)=g(n)+h(n)$.
4. Expand neighboring nodes.
5. Update costs if a better path is found.
6. Repeat until the goal node is reached.

Flowchart

Start

|

Initialize Open List

|

Goal Reached?

/ \

No Yes

|

Select Lowest Cost Node

|

Expand Neighbors

|

Update Costs

```
|  
Repeat  
|  
End
```

Program

```
graph = {  
    'A': [('B',1),('C',3)],  
    'B': [('D',3),('E',1)],  
    'C': [('F',5)],  
    'D': [],  
    'E': [('F',2)],  
    'F': []  
}  
  
heuristic = {  
    'A':5,  
    'B':4,  
    'C':3,  
    'D':2,  
    'E':1,  
    'F':0  
}  
  
open_list=[('A',0)]  
visited=[]  
  
while open_list:  
    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])  
    node,cost=open_list.pop(0)  
  
    if node in visited:  
        continue  
  
    print(node,end=" ")  
  
    if node=='F':
```

```
        break

    visited.append(node)

    for neigh,w in graph[node]:
        open_list.append((neigh,cost+w))
```

Output

A B E F

Result

The shortest path was found successfully using the A* algorithm.

Conclusion

A* efficiently finds the shortest path using both path cost and heuristic information.